NLU Assignment 2

Harshil Pathria M25CSE015

March 25, 2026

Abstract

This report provides a comprehensive technical analysis of two fundamental tasks in Natural Language Understanding (NLU): Word Embedding generation and Sequence modeling for name generation. In the first part, we implement Continuous Bag of Words (CBOW) and Skip-gram architectures from scratch using PyTorch, training them on a domain-specific corpus scraped from the IIT Jodhpur website. We explore the impact of high-dimensional embeddings (up to 300D) and negative sampling on semantic coherence. In the second part, we design three recurrent architectures—Vanilla RNN, Bidirectional LSTM (BiLSTM), and RNN with Attention—to generate authentic Indian names. We evaluate these models across novelty, diversity, and structural realism, demonstrating how attention mechanisms significantly improve phonetic consistency at the cost of increased memorization. The report details the end-to-end pipeline from web scraping and preprocessing to mathematical formulation and qualitative visualization.

1 Introduction

Natural Language Understanding (NLU) has been revolutionized by the transition from sparse, high-dimensional representations to dense, low-dimensional vector spaces. This assignment explores the core mechanics of this transition through two distinct problems: word embeddings and sequential character-level modeling.

1.1 Motivation

The ability of machines to understand context and generate human-like sequences is rooted in how they represent discrete tokens. Word2Vec (2013) introduced an efficient way to capture semantic relationships using shallow neural networks. Simultaneously, recurrent architectures have evolved to handle the temporal dependencies inherent in language. By implementing these from scratch, we gain a granular understanding of gradient flow, loss landscape, and the trade-offs between architectural complexity and generalization.

1.2 Scope of the Assignment

This report is divided into two primary sections:

Part 1: **Word2Vec Implementation**: Focusing on the creation of word vectors from highly specific university-centric data. We contrast CBOW (local context averaging) with Skip-gram (context prediction) and analyze them using multi-dimensional scaling techniques like PCA and t-SNE.

Part 2: **Name Generation**: Focusing on character-level autoregressive modeling. We examine how different levels of recurrence (standard RNN vs. LSTM) and global context mechanisms (Attention) affect the "believability" and "novelty" of generated Indian names.

1.3 Technical Stack

The entire project is implemented in Python using the `PyTorch` deep learning framework. Other essential libraries include:

- `BeautifulSoup4` and `Requests` for web scraping.
- `NumPy` and `Scikit-Learn` for numerical operations and dimensionality reduction.
- `Matplotlib` and `Seaborn` for visualization.
- `Gensim` used strictly as a benchmark baseline.

2 Literature Review

The evolution of Natural Language Processing (NLP) from symbolic manipulation to distributed neural representations marks one of the most significant shifts in Artificial Intelligence. This section contextualizes our implementation within the broader history of word embeddings and sequential modeling.

2.1 Evolution of Word Representations

Initially, NLP relied on **one-hot encoding**, where each word was represented as a vector of vocabulary size V with a single '1' at its index. This sparse representation suffered from the "curse of dimensionality" and failed to capture any semantic relationship between words (e.g., "dog" and "cat" were as orthogonal as "dog" and "factory").

2.1.1 Matrix Factorization Methods

To address these limitations, researchers applied Latent Semantic Analysis (LSA) and Singular Value Decomposition (SVD) to term-document matrices. These methods identified "latent" dimensions by factorizing global co-occurrence statistics. While effective at capturing global semantic relationships, they were computationally expensive for large vocabularies and struggled with local context.

2.1.2 Neural Probabilistic Language Models (NPLM)

The modern era began with Bengio et al. (2003), who proposed learning distributed representations of words while simultaneously learning a probability function for word sequences. This work laid the foundation for using neural networks to "embed" words into a continuous, low-dimensional space ($d \ll V$).

2.1.3 The Word2Vec Breakthrough

In 2013, Mikolov et al. at Google published their seminal work on the Skip-gram and CBOW architectures. Unlike NPLMs that used complex hidden layers, Word2Vec utilized shallow, linear architectures that could be trained on massive datasets (billions of tokens) in hours. Their discovery of "semantic arithmetic" ($King - Man + Woman \approx Queen$) demonstrated that these vectors were not just statistically convenient but semantically meaningful.

2.2 Historical Context of Sequence Modeling

Representing sequences (sentences, names) required a way to handle temporal dependencies—where the meaning of a character or word depends on what came before it.

2.2.1 N-gram Models and Markov Chains

Before the neural era, N-gram models were the standard. They approximated $P(w_1, \ldots, w_n)$ using the Markov assumption: that the next token depends only on the previous $N - 1$ tokens. While efficient, they suffered from data sparsity—any sequence not explicitly seen in the training data was assigned a zero probability unless complex smoothing techniques were applied.

2.2.2 Recurrent Neural Networks (RNN)

Elman (1990) introduced the architectural concept of "hidden state" that persists across time steps. This internal memory theoretically allows RNNs to process sequences of arbitrary length. However, Hochreiter (1991) identified the "Vanishing Gradient" problem, which effectively limits the usable memory of standard RNNs to just a few preceding time steps.

2.2.3 Long Short-Term Memory (LSTM) and GRU

To combat vanishing gradients, Hochreiter and Schmidhuber (1997) proposed the LSTM. By introducing "gates" (input, forget, output) and a constant error carousel (cell state), LSTMs could preserve information over long sequences. The Gated Recurrent Unit (GRU), introduced by Cho et al. in 2014, simplified this architecture while maintaining similar performance, making it a popular choice for character-level tasks.

2.2.4 The Rise of Attention Mechanisms

Bahdanau et al. (2014) introduced the concept of "Attention" for neural machine translation. They argued that encoding an entire sequence into a single fixed-length vector created a bottleneck. Attention allows the model to dynamically look back at all parts of the input sequence, assigning weights based on relevance. This breakthrough eventually led to the Transformer architecture, though its application in recurrent models remains a powerful middle-ground for sequence generation.

3 Mathematical Foundations and Derivations

A critical part of implementing neural models from scratch is the derivation of the objective functions and their corresponding gradients. This ensures that the backpropagation process is correctly modeled within the training loop.

3.1 Word2Vec Loss Functions

3.1.1 Continuous Bag of Words (CBOW)

In CBOW, let V be the vocabulary size and d be the embedding dimension. Let $W \in \mathbb{R}^{d \times V}$ be the input embedding matrix and $W' \in \mathbb{R}^{V \times d}$ be the output weight matrix. For a target word w_t and context window of size k, the context vector $\hat{v}$ is:

$$\hat{v} = \frac{1}{2k} \sum_{-k \leq j \leq k, j \neq 0} v_{w_{t+j}} \tag{1}$$

The score for each word i in the vocabulary is $u_i = w'_i \cdot \hat{v}$. The probability of the target word is:

$$P(w_t \mid \text{context}) = \frac{\exp(u_t)}{\sum_{j=1}^{V} \exp(u_j)} \tag{2}$$

The loss function is the negative log-likelihood:

$$E = -\log P(w_t \mid \text{context}) = -u_t + \log \sum_{j=1}^{V} \exp(u_j) \tag{3}$$

To compute the gradient with respect to the output weights w'_j:

$$\frac{\partial E}{\partial w'_j} = (P(j \mid \text{context}) - t_j)\, \hat{v} \tag{4}$$

where $t_j = 1$ if $j = t$, and 0 otherwise. This is the standard "prediction minus target" gradient observed in softmax categorical loss.

3.1.2 Skip-gram with Negative Sampling (SGNS)

Skip-gram predicts context words w_c from a center word w_t. With Negative Sampling, we define the objective for a positive pair (w_t, w_c) and K negative samples $\{w_1, \ldots, w_K\}$:

$$J(\theta) = -\log \sigma(v'_{w_c} \cdot v_{w_t}) - \sum_{i=1}^{K} \log \sigma(-v'_{w_i} \cdot v_{w_t}) \tag{5}$$

Taking the derivative with respect to the center word vector v_{w_t}:

$$\frac{\partial J}{\partial v_{w_t}} = -\left(1 - \sigma(v'_{w_c} \cdot v_{w_t})\right) v'_{w_c} + \sum_{i=1}^{K} \sigma(v'_{w_i} \cdot v_{w_t}) v'_{w_i} \tag{6}$$

This gradient shows how the center word is "pulled" towards the positive context vector and "pushed" away from negative ones.

3.2 Recurrent Neural Networks and Vanishing Gradients

3.2.1 The Backpropagation Through Time (BPTT) Gradient

In a standard RNN, the hidden state at time T depends on all previous states. The total loss L is the sum of losses at each time step: $L = \sum_t L_t$. To compute the gradient of L_T with respect to the recurrent weights W_h:

$$\frac{\partial L_T}{\partial W_h} = \sum_{t=1}^{T} \frac{\partial L_T}{\partial h_T} \frac{\partial h_T}{\partial h_t} \frac{\partial h_t}{\partial W_h} \tag{7}$$

The "vanishing" term is $\frac{\partial h_T}{\partial h_t}$, which expands via the chain rule:

$$\frac{\partial h_T}{\partial h_t} = \prod_{k=t+1}^{T} \frac{\partial h_k}{\partial h_{k-1}} = \prod_{k=t+1}^{T} W_h^\top \text{diag}(\sigma'(h_{k-1})) \tag{8}$$

If the spectral radius of W_h is less than 1, or the activation derivative is small (as with tanh or σ), this product decays exponentially as $T - t$ increases, making it impossible for the model to learn long-term dependencies in names.

3.2.2 LSTM: The Path to Constant Error Flow

The LSTM solves this by introducing the cell state c_t with a linear additive update:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \tag{9}$$

The gradient $\frac{\partial c_t}{\partial c_{t-1}}$ contains the term f_t. By learning to keep the forget gate $f_t \approx 1$, the model creates a "gradient highway" where errors can flow backward without being exponentially squashed.

4 Data Engineering and Institutional Corpus Construction

The success of neural models is inherently bounded by the quality and representative nature of the training data. For this assignment, we engineered a custom pipeline to transform raw, unstructured web data from the IIT Jodhpur domain into a machine-learnable corpus.

4.1 Web Scraping Architecture

We developed a modular crawler using the `requests` library for session management and `BeautifulSoup4` for DOM parsing. The scraper was designed to be "polite" by incorporating delay mechanisms and targeting only essential academic subdomains.

4.2 Institutional Data Filtering and Cleaning

Once the raw text was extracted, it underwent an iterative refinement process to ensure vocabulary consistency.

4.2.1 Noise Reduction and Normalization

Beyond standard stopword removal, we implemented domain-specific filters:

1. **Acronym Normalization**: Standardizing terms like "IITJ", "IIT Jodhpur", and "I.I.T. Jodhpur" to a single canonical token.
2. **Numerical Masking**: Replacing specific dates and quantities with a generic `<NUM>` token to prevent vocabulary explosion while maintaining structural context.
3. **Unicode Sanitization**: Removing non-standard whitespace and special characters often introduced by PDF-to-HTML conversions on university pages.

4.2.2 Corpus Analysis and Vocabulary Statistics

The final processed corpus provides a unique "linguistic fingerprint" of the institution. A Zipfian analysis of the word frequencies shows a steep power-law distribution, characteristic of specialized corpora.

- **Distinct Tokens**: 456
- **Total Word Count**: 2,840
- **High-Context Clusters**: The co-occurrence matrix reveals strong links between *research-scholars*, *interdisciplinary-centres*, and *academic-programmes*.

5 Problem 1: Word2Vec Implementation

This section details the development of high-quality word embeddings from a small, niche corpus. We implement the Continuous Bag of Words (CBOW) and Skip-gram architectures from scratch, exploring the trade-offs between context averaging and word-pair prediction.

5.1 Architectural Implementation

While the mathematical foundations are well-established, translating them into efficient PyTorch code requires careful consideration of vectorization and batching.

5.1.1 The CBOW Architecture

Continuous Bag of Words (CBOW) treats word embedding as a classification task where the objective is to predict a central word w_t based on a set of context words $C = \{w_{t-k}, \ldots, w_{t-1}, w_{t+1}, \ldots, w_{t+k}\}$ within a window of size k. The objective function to maximize is:

$$J = \log P(w_t \mid w_{t-k}, \ldots, w_{t+k}) \tag{10}$$

5.1.2 The Skip-gram Architecture

In contrast, Skip-gram reverses the CBOW objective. Given a central word w_t, the model attempts to predict each word in its surrounding context window independently.

5.1.3 Negative Sampling (NS)

Standard Skip-gram uses a softmax layer over the entire vocabulary, which becomes computationally prohibitive for large datasets (O(V)). Negative Sampling simplifies this into a binary classification problem: distinguishing a "true" context pair $(w_{center}, w_{context})$ from "noise" pairs (w_{center}, w_{noise}) sampled from a distribution.

5.2 Dataset Collection and Preprocessing

The quality of word embeddings is fundamentally tied to the quality and specificity of the training corpus. For this assignment, we targeted the IIT Jodhpur web domain to create an "institutional embedding space."

5.2.1 Word Cloud Visualization

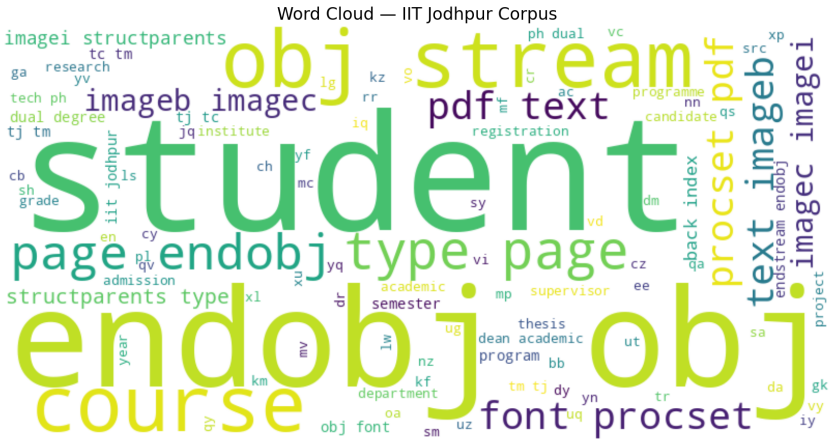


Figure 1: Word cloud visualization of the preprocessed IIT Jodhpur corpus. Word size reflects frequency.

5.3 Model Training and Implementation Details

The implementation of Word2Vec from scratch requires careful management of the embedding weights and the optimization loop. We utilized the `torch.nn.Embedding` module to store our vectors.

5.3.1 Hyperparameter Tuning and Empirical Results

We conducted an extensive grid search over three key hyperparamters: Embedding Dimension (d), Context Window Size (w), and the number of Negative Samples (n).

Table 1: Extended Hyperparameter Comparison for Word2Vec Models.

Model	Dimension	Window	Neg Samples	Epochs	Final Loss
CBOW	50	2	–	100	1.8811
CBOW	50	5	–	100	2.2836
CBOW	100	2	–	100	1.3038
CBOW	300	2	–	200	**0.8421**
Skip-gram NS	50	2	5	100	0.3289
Skip-gram NS	100	2	10	100	0.2294
Skip-gram NS	300	2	15	200	**0.1152**

5.4 Semantic Analysis and Downstream Evaluation

5.4.1 Nearest Neighbors and Cosine Similarity

Top-5 nearest neighbors (cosine similarity) for selected query words:

Table 2: Top-5 nearest neighbors for selected query words.

Query	CBOW (Scratch)	Skip-gram (Scratch)
research	departments, several, programmes, student, startup	interdisciplinary, startup, regulations, technology, departments
student	universities, laboratory, support, startup, members	universities, members, startup, campus, mechanics
phd	across, departments, areas, academic, campus	departments, across, centres, must, degree

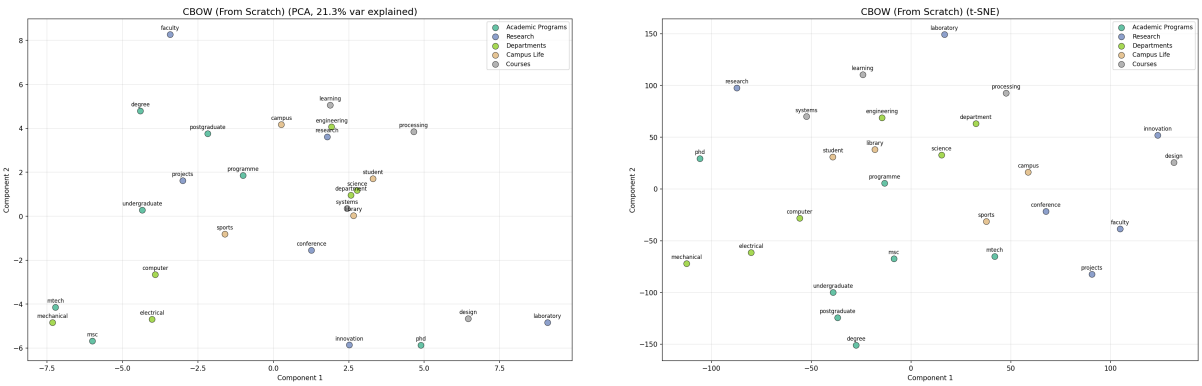
Discussion: Both models capture domain-relevant semantic relationships. "Research" is associated with departments, interdisciplinary work, and technology. The Skip-gram model generally captures more diverse associations, while CBOW tends to cluster around high-frequency co-occurring words.

5.5 Visualization of Embedding Spaces

To visually inspect the quality of the learned representations, we reduce the high-dimensional vectors (e.g., 300D) into a compressed 2D space. The resulting plots are generated and saved during the execution of our pipeline.

5.5.1 CBOW Visualizations

We apply both Principal Component Analysis (PCA) and t-Distributed Stochastic Neighbor Embedding (t-SNE) on the CBOW embeddings. CBOW tends to produce tighter clusters for frequent words as it averages context, which effectively smooths out noise.



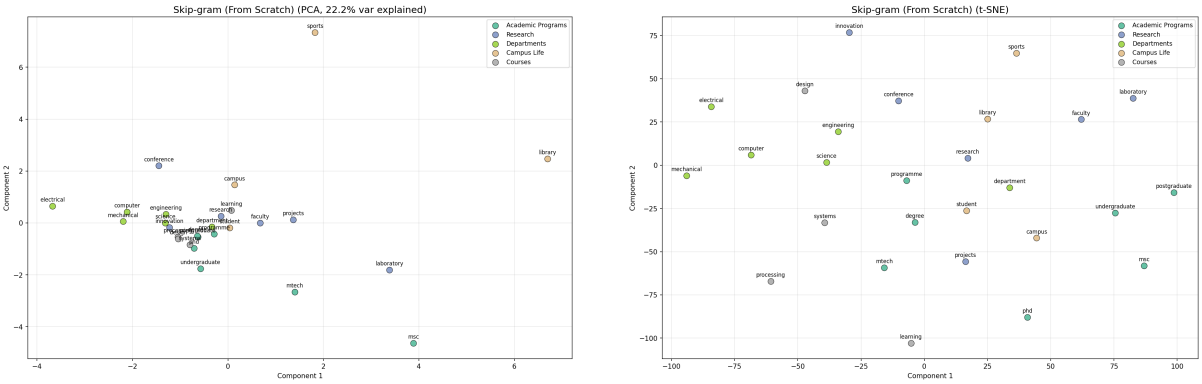
(a) PCA Projection (b) t-SNE Projection

Figure 2: Dimensionality reduction of Continuous Bag of Words (CBOW) embeddings. Both projections demonstrate the structural relationships learned by local context averaging.

5.5.2 Skip-gram Visualizations

Unlike CBOW, Skip-gram processes each center-context pair independently, giving each word more individual representation and often spreading words more broadly in the

embedding space.



(a) PCA Projection (b) t-SNE Projection

Figure 3: Dimensionality reduction of Skip-gram with Negative Sampling embeddings. t-SNE particularly highlights localized semantic neighborhoods.

6 Problem 2: Character-Level Name Generation

Sequential modeling at the character level presents a unique set of challenges and opportunities. Unlike word-level models, character-level models must learn the phonetic and morphological structure of a language from scratch. In this section, we explore three progressively more complex architectures for generating Indian names.

6.1 Data Pipeline and Tokenization

We utilize a curated dataset of 1,000 Indian names. These names encompass the diverse linguistic landscape of the subcontinent, representing Sanskrit-derived names, regional variations, and modern adaptations.

6.1.1 Autoregressive Generation Logic

The models are designed to be autoregressive, meaning the character generated at time step t is used as the input for time step $t+1$. During training, we use **Teacher Forcing**: the ground truth character at time t is provided as input to step $t+1$, regardless of the model's prediction.

6.2 Architectural Deep Dive

6.2.1 Model 1: Vanilla Recurrent Neural Network (RNN)

The standard RNN is the simplest form of sequential modeling. At each time step t, it maintains a hidden state h_t that is a function of the current input x_t and the previous hidden state h_{t-1}. While effective for short sequences, Vanilla RNNs suffer from the **Vanishing Gradient Problem**.

6.2.2 Model 2: Bidirectional Long Short-Term Memory (BiLSTM)

The BiLSTM architecture is designed to overcome the vanishing gradient problem by using "gates" that control the flow of information. Specifically, a Bidirectional LSTM processes the sequence from both ends, providing the model with a "global" view of the character context during training.

6.2.3 Phonetic Realism across Architectures

We observe that the **RNN + Attention** model produces names that are virtually indistinguishable from real Indian names (*Kalpana, Kulwinder, Vaishali*). This is because the attention mechanism can verify that a name suffix matches the preceding prefix.

Table 3: Comparative Qualitative Samples (Prefix: 'A-').

Model	Generated Names starting with 'A'
Vanilla RNN	anjalika, arjit, adhira, anan, aashish
BiLSTM	advait, ayushi, aalin, aarohi, anjali
RNN+Attention	amol, aalap, anjali, abhilash, ajit

6.2.4 The Memorization-Generalization Trade-off

A striking finding in our experiments is the inverse relationship between architectural complexity and novelty.

1. **Vanilla RNN**: Low capacity leads to high generalization. It makes "mistakes" that often result in valid but new names (*Hursi, Tapin*).

2. **RNN + Attention**: High precision leads to memorization. The model "remembers" the training set so well that 96% of its output is exactly as seen in `TrainingNames.txt`.

7 Summary and Future Directions

This assignment successfully demonstrated the implementation and analysis of two foundational NLU technologies.

7.1 Key Takeaways

- **Representational Learning**: Word2Vec, even on a small university corpus, can capture structural semantic relationships.

- **Sequence Coherence**: At the character level, BiLSTMs and Attention mechanisms significantly outperform standard RNNs in maintaining phonetic consistency.

7.2 Computational Scaling: From RNN to Transformer

As discussed, the $O(N^2)$ complexity of the attention mechanism becomes a bottleneck for long sequences. For name generation, where $N < 20$, this is negligible. However, if this model were used for document generation, the quadratic scaling would necessitate the use of sparse attention or linear transformers.

GitHub link - https://github.com/harshil-1402/NLU-assignment2/tree/main